

ArrayBlockingQueue<E>

Fabrice.Kordon@lip6.fr



Ce que fait ArrayBlockingQueue

2

Gestion d'une file de E

- Ordonnancement FIFO
 - ▶ La «tête» = plus ancien élément dans la file
 - ▶ Ma «queue» = plus récent élément dans la file
- Thread-safe
- Paramétrage
 - ▶ La taille de la FIFO
 - ▶ Politique d'équité
- Implémente les interfaces suivantes
 - ▶ Collection
 - ▶ Iterator

AAAARG!!! on aurait pu ne pas faire la file «à la mano» dans les exemples précédents?



Par exemple...



- La file de requêtes du serveur multithreadé «gratuitement»
 - ▶ Mais au moins on a compris comment faire

Quelques primitives utile

3

Constructeurs

- `ArrayBlockingQueue(int capacity)`
- `ArrayBlockingQueue(int capacity, boolean fair)`
- `ArrayBlockingQueue(int capacity, boolean fair, Collection<? extends E> c)`

Gestion

- `int size()`
- `int remainingCapacity()`



Sur la primitive bloquante
Rattraper `InterruptedException`

Ajout d'éléments

- `boolean add(E e)`
 - `boolean offer(E e)`
 - `boolean offer(E e, long timeout, TimeUnit unit)`
 - `void put(E e)`
- non bloquant
- bloquant

Autres primitives utiles...

4

Extraction d'éléments

- `void clear()`
- `int drainTo(Collection<? super E> c)`
- `int drainTo(Collection<? super E> c, int maxElements)`

- `E peek()`
- `E poll()`
- `E poll(long timeout, TimeUnit unit)`

→ non bloquant

- `E take()`

→ bloquant

 **Sur la primitive bloquante**
Rattraper `InterruptedException`



En guise de conclusion...

5

 **Cela va servir...**

-  Pour réécrire le serveur multi-threadé

